

Experiment 4

Aim: Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

Source Code:

```
-- -----  
-- 1. CREATE TABLES  
-- -----  
  
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50),  
    location VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary DECIMAL(10,2),  
    job_title VARCHAR(50),  
    dept_id INT,  
    manager_id INT NULL,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),  
    FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)  
);  
  
CREATE TABLE Project (  
    project_id INT PRIMARY KEY,  
    project_name VARCHAR(100),  
    dept_id INT,  
    budget DECIMAL(12,2),  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Employee_Project (  
    emp_id INT,  
    project_id INT,  
    hours_worked INT,
```

```

PRIMARY KEY (emp_id, project_id),
FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),
FOREIGN KEY (project_id) REFERENCES Project(project_id)
);

-- =====

-- 2. INSERT DATA

-- =====

INSERT INTO Department VALUES
(1, 'IT', 'Delhi'),
(2, 'HR', 'Mumbai'),
(3, 'Finance', 'Bangalore'),
(4, 'Marketing', 'Pune'),
(5, 'AI/ML', 'Hyderabad');

INSERT INTO Employee VALUES
(101, 'Rahul', 75000, 'Developer', 1, NULL),
(102, 'Aman', 65000, 'Developer', 1, 101),
(103, 'Priya', 90000, 'Manager', 2, NULL),
(104, 'Neha', 60000, 'HR Executive', 2, 103),
(105, 'Rohit', 85000, 'Analyst', 3, NULL),
(106, 'Sneha', 55000, 'Accountant', 3, 105),
(107, 'Karan', 70000, 'Marketing Executive', 4, NULL),
(108, 'Pooja', 50000, 'Marketing Executive', 4, 107),
(109, 'Arjun', 95000, 'AI Engineer', 5, NULL),
(110, 'Simran', 80000, 'ML Engineer', 5, 109),
(111, 'Vikas', 62000, 'Developer', 1, 101),
(112, 'Anjali', 58000, 'Developer', 1, 101),
(113, 'Mohit', 72000, 'HR Executive', 2, 103),
(114, 'Riya', 68000, 'Analyst', 3, 105),
(115, 'Varun', 78000, 'AI Engineer', 5, 109),
(116, 'Tanya', 52000, 'Marketing Executive', 4, 107),
(117, 'Aditya', 88000, 'Developer', 1, 101),
(118, 'Nisha', 61000, 'Accountant', 3, 105),
(119, 'Yash', 73000, 'ML Engineer', 5, 109),
(120, 'Kriti', 57000, 'HR Executive', 2, 103),

```

(121, 'Dev', 67000, 'Developer', 1, 101),
(122, 'Isha', 54000, 'Marketing Executive', 4, 107),
(123, 'Manish', 91000, 'AI Engineer', 5, 109),
(124, 'Ayesha', 63000, 'Analyst', 3, 105),
(125, 'Kabir', 76000, 'Developer', 1, 101),
(126, 'Meera', 59000, 'HR Executive', 2, 103),
(127, 'Sahil', 69000, 'Accountant', 3, 105),
(128, 'Diya', 81000, 'ML Engineer', 5, 109),
(129, 'Harsh', 71000, 'Marketing Executive', 4, 107),
(130, 'Muskan', 56000, 'HR Executive', 2, 103);

INSERT INTO Project VALUES

(201, 'Website Development', 1, 500000),
(202, 'Mobile App', 1, 700000),
(203, 'Recruitment System', 2, 300000),
(204, 'Financial Analysis', 3, 600000),
(205, 'Advertisement Campaign', 4, 400000),
(206, 'AI Chatbot', 5, 900000),
(207, 'Machine Learning Model', 5, 1000000),
(208, 'Data Analytics', 3, 800000);

INSERT INTO Employee_Project VALUES

(101,201,120),
(102,201,100),
(111,202,90),
(112,202,80),
(117,201,110),
(125,202,100),
(103,203,70),
(104,203,90),
(113,203,80),
(120,203,60),
(105,204,100),
(106,204,90),
(114,208,110),
(118,208,80),

(124,204,70),
 (127,208,90),
 (107,205,100),
 (108,205,80),
 (116,205,70),
 (122,205,60),
 (129,205,90),
 (109,206,120),
 (110,206,100),
 (115,207,110),
 (119,207,100),
 (123,206,90),
 (128,207,120);
 SELECT * FROM Employee;

The screenshot shows the byteXL SQL editor interface. The left sidebar contains navigation links: Home, Dashboard, Feedback Requests, Reports, Student Reports, Learning, AI Mentor, AI Interviewer, Learn, Editor (highlighted), Lab, Assessment, and Nimbus. The main editor area displays a SQL script with line numbers 82 to 112. The script includes multiple INSERT statements into the Employee_Project table and a final SELECT * FROM Employee; statement. The right panel shows the output of the query, which is a table with 5 columns: emp_id, emp_name, salary, job_title, and dept. The output table contains 18 rows of data.

emp_id	emp_name	salary	job_title	dept
101	Rahul	75000.00	Developer	
102	Aman	65000.00	Developer	
103	Priya	90000.00	Manager	
104	Neha	60000.00	HR Executive	
105	Rohit	85000.00	Analyst	
106	Sneha	55000.00	Accountant	
107	Karan	70000.00	Marketing Executive	
108	Pooja	50000.00	Marketing Executive	
109	Arjun	95000.00	AI Engineer	
110	Samaran	80000.00	ML Engineer	
111	Vikas	62000.00	Developer	
112	Anjali	58000.00	Developer	
113	Mohit	72000.00	HR Executive	
114	Riya	68000.00	Analyst	
115	Varun	78000.00	AI Engineer	
116	Tanya	52000.00	Marketing Executive	
117	Aditya	88000.00	Developer	
118	Nisha	61000.00	Accountant	

OUTPUT:

INNER JOIN:

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id;

The screenshot shows the byteXL IDE interface. On the left, a query editor contains a MySQL query that inserts data into an 'Employee' table and then performs a self-join. The query is as follows:

```
79 (205, 'Advertisement Campaign', 4, 400000),
80 (206, 'AI Chatbot', 5, 900000),
81 (207, 'Machine Learning Model', 5, 1000000),
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (110,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (100,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (128,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name, d.dept_name
113 FROM Employee e
114 INNER JOIN Department d
115 ON e.dept_id = d.dept_id;
```

On the right, the output window displays a table with the following columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. The table contains 28 rows of data, including employees like Rahul, Aman, Priya, Neha, Rohit, Sneha, Karan, Pooja, Arjun, and others, along with their respective salaries, job titles, and department IDs.

LEFT JOIN:

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

LEFT JOIN Department d

ON e.dept_id = d.dept_id;

The screenshot shows the byteXL IDE interface. On the left, a query editor contains a MySQL query that inserts data into an 'Employee' table and then performs a self-join. The query is as follows:

```
79 (205, 'Advertisement Campaign', 4, 400000),
80 (206, 'AI Chatbot', 5, 900000),
81 (207, 'Machine Learning Model', 5, 1000000),
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (110,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (100,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (128,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name, d.dept_name
113 FROM Employee e
114 LEFT JOIN Department d
115 ON e.dept_id = d.dept_id;
```

On the right, the output window displays a table with the following columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. The table contains 28 rows of data, including employees like Rahul, Aman, Priya, Neha, Rohit, Sneha, Karan, Pooja, Arjun, and others, along with their respective salaries, job titles, and department IDs.

SELF JOIN:

SELECT

e.emp_name AS Employee, m.emp_name AS Manager

FROM Employee e

LEFT JOIN Employee m

ON e.manager_id = m.emp_id;

```
80 (200, 'AI Chatbot', 5, 900000),
81 (207, 'Machine Learning Model', 5, 1000000),
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (118,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (108,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT
113 e.emp_name AS Employee, m.emp_name AS Manager
114 FROM Employee e
115 LEFT JOIN Employee m
116 ON e.manager_id = m.emp_id;
```

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	58000.00	Developer	1	101
113	Rohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	89000.00	Accountant	3	105

THREE-WAY JOIN:

SELECT e.emp_name, d.dept_name, p.project_name, ep.hours_worked

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id

INNER JOIN Employee_Project ep

ON e.emp_id = ep.emp_id

INNER JOIN Project p

ON ep.project_id = p.project_id;

```
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (118,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (108,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_name, d.dept_name, p.project_name, ep.hours_worked
113 FROM Employee e
114 INNER JOIN Department d
115 ON e.dept_id = d.dept_id
116 INNER JOIN Employee_Project ep
117 ON e.emp_id = ep.emp_id
118 INNER JOIN Project p
119 ON ep.project_id = p.project_id;
120
```

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	58000.00	Developer	1	101
113	Rohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	89000.00	Accountant	3	105

CORRELATED SUBQUERY:

```
SELECT e.emp_id, e.emp_name, e.salary, e.dept_id
```

```
FROM Employee e
```

```
WHERE e.salary > ( SELECT AVG(e2.salary)
```

```
FROM Employee e2
```

```
WHERE e2.dept_id = e.dept_id );
```

The screenshot shows the byteXL SQL editor with a MySQL database. The query in the editor is:

```
98 (200, 'AI Chatbot', 3, 8000000),
99 (201, 'Machine Learning Model', 5, 10000000),
100 (202, 'Data Analytics', 3, 8000000);
101 INSERT INTO Employee_Project VALUES
102 (101,201,120),
103 (102,201,100),
104 (111,202,90),
105 (112,202,200),
106 (117,201,110),
107 (125,202,100),
108 (103,203,70),
109 (104,203,90),
110 (113,203,80),
111 (120,203,60),
112 (105,204,100),
113 (106,204,90),
114 (114,200,110),
115 (110,200,80),
116 (124,204,70),
117 (127,200,90),
118 (107,205,100),
119 (100,205,80),
120 (130,205,70),
121 (122,205,60),
122 (129,205,90),
123 (109,206,120),
124 (110,206,100),
125 (115,207,110),
126 (119,207,100),
127 (123,206,90),
128 (120,207,120);
129 SELECT * FROM Employee;
130 SELECT e.emp_id, e.emp_name, e.salary, e.dept_id
131 FROM Employee e
132 WHERE e.salary > ( SELECT AVG(e2.salary)
133 FROM Employee e2
134 WHERE e2.dept_id = e.dept_id );
```

The output table shows the following data:

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	50000.00	Developer	1	101
113	Mohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	69000.00	Accountant	3	105

EXISTS:

```
SELECT e.emp_id, e.emp_name
```

```
FROM Employee e
```

```
WHERE EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The screenshot shows the byteXL SQL editor with a MySQL database. The query in the editor is:

```
78 'Financial Analysis', 3, 8000000),
79 'Advertisement Campaign', 4, 4000000),
80 'AI Chatbot', 3, 8000000),
81 'Machine Learning Model', 5, 10000000);
82 INSERT INTO Employee_Project VALUES
83 (101,201,120),
84 (102,201,100),
85 (111,202,90),
86 (112,202,200),
87 (117,201,110),
88 (125,202,100),
89 (103,203,70),
90 (104,203,90),
91 (113,203,80),
92 (120,203,60),
93 (105,204,100),
94 (106,204,90),
95 (114,200,110),
96 (110,200,80),
97 (124,204,70),
98 (127,200,90),
99 (107,205,100),
100 (100,205,80),
101 (130,205,70),
102 (122,205,60),
103 (129,205,90),
104 (109,206,120),
105 (110,206,100),
106 (115,207,110),
107 (119,207,100),
108 (123,206,90),
109 (120,207,120);
110 SELECT * FROM Employee;
111 SELECT e.emp_id, e.emp_name
112 FROM Employee e
113 WHERE EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The output table shows the following data:

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	50000.00	Developer	1	101
113	Mohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	69000.00	Accountant	3	105
128	Diya	81000.00	ML Engineer	5	109
129	Harsh	71000.00	Marketing Executive	4	107
130	Muskan	56000.00	HR Executive	2	103

SIMULATED INTERSECT:

```
SELECT e.emp_id, e.emp_name
```

```
FROM Employee e
```

```
WHERE e.salary > 70000
```

```
AND EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The screenshot shows the byteXL SQL editor interface. The left pane displays a SQL query with line numbers 79 to 115. The query is as follows:

```
79 (205, 'Advertisement Campaign', 4, 400000),
80 (206, 'AI Chatbot', 5, 900000),
81 (207, 'Machine Learning Model', 5, 1000000),
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (110,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (100,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (118,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name
113 FROM Employee e
114 WHERE e.salary > 70000
115 AND EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The right pane shows the output of the query, which is a table with 6 columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. The table contains 130 rows of data, including employees like Rahul, Aman, Priya, Neha, Rohit, Sneha, Karan, Pooja, Arjun, Vikas, Anjali, Mohit, Riya, Varun, Tanya, Aditya, Nisha, Yash, Kriti, Dev, Isha, Manish, Ayesha, Kabir, Meera, Sahil, Diya, Harsh, and Muskan.

SIMULATED EXCEPT:

```
SELECT e.emp_id, e.emp_name
```

```
FROM Employee e
```

```
WHERE NOT EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The screenshot shows the byteXL SQL editor interface. The left pane displays a SQL query with line numbers 78 to 113. The query is as follows:

```
78 (204, 'Financial Analysis', 3, 600000),
79 (205, 'Advertisement Campaign', 4, 400000),
80 (206, 'AI Chatbot', 5, 900000),
81 (207, 'Machine Learning Model', 5, 1000000),
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (110,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (100,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (118,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name
113 FROM Employee e
114 WHERE NOT EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The right pane shows the output of the query, which is a table with 6 columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. The table contains 130 rows of data, including employees like Rahul, Aman, Priya, Neha, Rohit, Sneha, Karan, Pooja, Arjun, Vikas, Anjali, Mohit, Riya, Varun, Tanya, Aditya, Nisha, Yash, Kriti, Dev, Isha, Manish, Ayesha, Kabir, Meera, Sahil, Diya, Harsh, and Muskan.

EXPLAIN:

EXPLAIN

SELECT e.emp_name, d.dept_name

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id;

The screenshot shows the ByteXL SQL editor interface. The left pane displays a SQL query with line numbers 80 to 116. The query is as follows:

```
80 (200, 'AI Chatbot', 5, 9000000),
81 (207, 'Machine Learning Model', 5, 10000000),
82 (208, 'Data Analytics', 3, 8000000);
83 INSERT INTO Employee_Project VALUES
84 (181,201,120),
85 (182,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (101,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,204,110),
97 (116,204,80),
98 (124,204,70),
99 (127,204,90),
100 (107,205,100),
101 (108,205,80),
102 (110,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (118,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 EXPLAIN
113 SELECT e.emp_name, d.dept_name
114 FROM Employee e
115 INNER JOIN Department d
116 ON e.dept_id = d.dept_id;
```

The right pane shows the execution plan (STON) and the output of the query. The output is a table with 5 columns: emp_id, emp_name, salary, job_title, and dept_id. The table contains 29 rows of employee data.

emp_id	emp_name	salary	job_title	dept_id
101	Rahul	75000.00	Developer	1
102	Aman	65000.00	Developer	1
103	Priya	90000.00	Manager	2
104	Neha	60000.00	HR Executive	2
105	Rohit	85000.00	Analyst	3
106	Sneha	55000.00	Accountant	3
107	Karan	70000.00	Marketing Executive	4
108	Pooja	50000.00	Marketing Executive	4
109	Arjun	95000.00	AI Engineer	5
110	Sinran	80000.00	ML Engineer	5
111	Vikas	62000.00	Developer	1
112	Anjali	58000.00	Developer	1
113	Mohit	72000.00	HR Executive	2
114	Riya	68000.00	Analyst	3
115	Varun	78000.00	AI Engineer	5
116	Tanya	52000.00	Marketing Executive	4
117	Aditya	88000.00	Developer	1
118	Nisha	61000.00	Accountant	3
119	Yash	73000.00	ML Engineer	5
120	Kriti	57000.00	HR Executive	2
121	Dev	67000.00	Developer	1
122	Isha	54000.00	Marketing Executive	4
123	Harish	91000.00	AI Engineer	5
124	Ayesha	63000.00	Analyst	3
125	Kabir	76000.00	Developer	1
126	Heera	59000.00	HR Executive	2
127	Sahil	69000.00	Accountant	3
128	Diya	81000.00	ML Engineer	5
129	Harsh	71000.00	Marketing Executive	4